Crop Recommendation System Using Machine Learning, Explainable AI and Fertilizer Recommendation

Author One
Dept. of Computer Science and Engineering
University Name
City, India
author1@example.com

Author Two
Dept. of Computer Science and Engineering
University Name
City, India
author2@example.com

Guide Name
Dept. of Computer Science and Engineering
University Name
City, India
guide@example.com

***Abstract*—Choosing a crop that suits the soil and the local weather is one of the first decisions a farmer makes each season, and a poor choice costs a full season of work. We built a crop recommendation system that takes seven inputs (soil nitrogen, phosphorus, potassium and pH, temperature, humidity and rainfall), recommends the three most suitable crops, explains the recommendation, and suggests how to correct the soil with fertilizer. We trained and tested ten machine learning algorithms on a public dataset of 2,200 samples covering 22 crops, using an 80/20 stratified split. Random Forest and Naive Bayes gave the highest test accuracy, 99.55% (438 of 440 test samples), followed by AdaBoost with depth-3 trees (99.32%) and XGBoost (99.09%). AdaBoost with its default settings reached only 17.50%, and we explain why. To show the farmer why a crop was chosen, we used SHAP and LIME on the Random Forest. Humidity (22%) and rainfall (18%) had the largest overall influence, and the two methods named the same top feature for 66% of 100 test cases. For fertilizer recommendation we compared eight algorithms on a 99-sample fertilizer dataset; XGBoost was best with 97.69% accuracy under repeated cross-validation, and the soil N, P and K values carried almost all of the predictive weight. A simple nutrient-gap rule then compares the farmer's soil with the needs of the recommended crop.**

***Index Terms*—Crop recommendation, machine learning, Random Forest, explainable AI, SHAP, LIME, fertilizer recommendation, precision agriculture**

I. Introduction

Most farmers in India decide what to sow from experience and from what their neighbours grow. That works when conditions stay the same, but soil nutrients change with years of cropping and fertilizer use, and rainfall varies from season to season. A crop that does not suit the field gives a low yield, and the loss is felt for the whole season.

Soil testing has become more common, and a farmer can now know the nitrogen (N), phosphorus (P) and potassium (K) content and the pH of the field. Weather data are also easy to obtain. The difficulty is turning these numbers into a decision. Machine learning is a natural fit here: given enough examples of the conditions under which each crop grows well, a model can learn which crop matches a new set of soil and weather values [1].

A recommendation alone is not enough, though. A farmer is unlikely to trust a crop name that comes with no reason, and the next question after "what should I grow?" is usually "what fertilizer should I apply?". We therefore designed the system around three parts: a crop recommender, an explanation of each recommendation, and a fertilizer module.

Our contributions are:

- A comparison of ten algorithms (Logistic Regression, Naive Bayes, K-Nearest Neighbours, SVM, Decision Tree, Random Forest, XGBoost, LightGBM and two AdaBoost variants) on the same train/test split, reporting accuracy, precision, recall, F1-score, cross-validation accuracy and training time.
- An explainable AI module that uses SHAP and LIME to show which soil and weather values led to a recommendation, both for the whole dataset and for a single farmer.
- A fertilizer module with two parts: a comparison of eight algorithms that predict the fertilizer from soil and crop data, and a nutrient-gap rule that compares the farmer's N, P and K with the requirement of the recommended crop.
- All results come from public datasets, which we list with their links so that others can repeat the work.

The rest of the paper is organised as follows. Section II covers the algorithms and related work. Section III describes the datasets and the proposed system. Section IV presents the results, and Section V concludes.

II. Background

A. Preliminary Concepts

1) Decision Tree and Random Forest: A decision tree splits the data on one feature at a time, choosing at each node the threshold that best separates the classes, until each leaf holds mostly one class [10]. A single tree is easy to read but changes a lot when the training data change slightly. Random Forest [11] trains many trees, each on a bootstrap sample of the data and with a random subset of features at each split, and takes the majority vote:

$$\hat{y} = \mathrm{mode}\{h_t(\mathbf{x})\}_{t=1}^{T}, \tag{1}$$

where h_t is the t-th tree. The fraction of trees that vote for each crop also gives a probability, which we use to rank crops.

2) K-Nearest Neighbours (KNN): KNN [12] stores the training samples and labels a new sample by a vote among the k samples closest to it. Closeness is measured with the Euclidean distance

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{j=1}^{M} (x_j - x'_j)^2}, \tag{2}$$

so features must be put on a common scale first; otherwise rainfall, measured in hundreds of millimetres, would dominate pH.

3) Naive Bayes: Gaussian Naive Bayes assumes that, within each crop, every feature follows a normal distribution and the features are independent. It picks the crop c that maximises

$$P(c) \prod_{j=1}^{M} \mathcal{N}\left(x_j;\, \mu_{cj}, \sigma_{cj}^2\right), \tag{3}$$

where μ_{cj} and σ_{cj} are the mean and standard deviation of feature j for crop c.

4) Support Vector Machine (SVM): An SVM finds the boundary with the widest margin between classes [13]. With a radial basis function (RBF) kernel the boundary can be curved, and multi-class problems are handled by combining several two-class SVMs.

5) Boosting: Boosting trains models one after another, each one focusing on the samples the previous ones got wrong. AdaBoost [14] raises the weight of misclassified samples after every round; its multi-class version is SAMME [15]. XGBoost [16] and LightGBM [17] are gradient boosting libraries that fit each new tree to the errors of the current ensemble and are widely used for tabular data.

6) SHAP: SHapley Additive exPlanations [18] split one prediction into a contribution from each feature:

$$f(\mathbf{x}) = \phi_0 + \sum_{j=1}^{M} \phi_j, \tag{4}$$

where ϕ_0 is the average output of the model and ϕ_j is the contribution of feature j, computed as a Shapley value from game theory. For tree models, TreeSHAP computes these values exactly and quickly [19].

7) LIME: LIME [20] explains one prediction by creating many slightly changed copies of the input, asking the model to predict them, and fitting a simple linear model g to those predictions:

$$\xi(\mathbf{x}) = \arg\min_{g} \; \mathcal{L}(f, g, \pi_{\mathbf{x}}) + \Omega(g), \tag{5}$$

where $\pi_{\mathbf{x}}$ gives more weight to copies close to $\mathbf{x}$ and Ω keeps g simple. The weights of g are the explanation.

B. Related Work

Liakos et al. [1] reviewed machine learning in agriculture and grouped the work into crop, livestock, water and soil management; crop recommendation sits in the first group. Van Klompenburg et al. [2] reviewed crop yield prediction studies; temperature, rainfall and soil type were among the features used most often.

For recommending a crop, Kumar et al. [3] proposed a crop selection method that chooses a sequence of crops to improve the yield rate over a season. Pudumalar et al. [4] combined several classifiers in an ensemble that recommends a crop from soil properties. Doshi et al. [5] built AgroConsultant, which compares several classifiers for crop recommendation using soil and climate inputs. The public Harvestify project [7] offers crop recommendation, fertilizer suggestion and plant disease detection on one website and is the source of the crop dataset we use.

Explainable AI has started to appear in agricultural data analysis [6]. The crop recommendation studies above compare the accuracy of their models but do not explain individual recommendations to the farmer, and fertilizer advice is treated as a separate problem. Our system brings the three steps together and compares a wider set of algorithms on the same split.

III. Proposed System

A. System Overview

Fig. 1 shows the system. The farmer supplies soil test values and weather conditions. The crop model returns the three most likely crops with a probability for each. The explanation module shows which inputs pushed the model towards the first crop. The fertilizer module then checks the soil against that crop's needs.

B. Datasets

We used three public datasets (Table I).

The crop dataset was released on Kaggle by the creator of Harvestify [7], [8]. It contains 22 crops, among them rice, maize, chickpea, cotton, jute, coffee and several fruits, with exactly 100 samples per crop. The fertilizer dataset [9] maps soil and crop information to one of seven fertilizers: Urea (22 rows), DAP (18), 28-28 (17), 14-35-14 (14), 20-20 (14), 17-17-17 (7) and 10-26-26 (7). The requirement table, taken from the Harvestify repository, lists the N, P and K values that each of the 22 crops needs, on the same scale as the crop dataset.

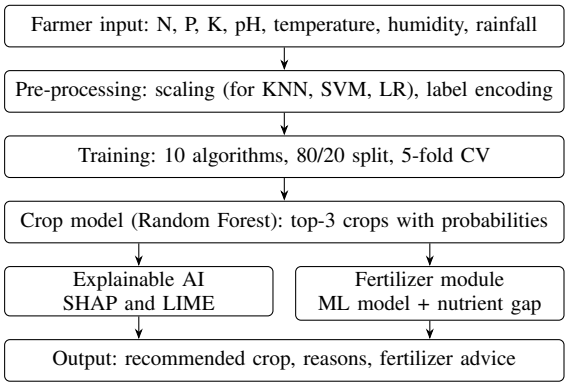


Fig. 1. Architecture of the proposed crop recommendation system.

TABLE I
DATASETS USED IN THIS WORK

Dataset	Rows	Classes	Features
Crop Recommendation [8]	2,200	22 crops (100 each)	N, P, K, temperature, humidity, pH, rainfall
Fertilizer Prediction [9]	99	7 fertilizers	temperature, humidity, moisture, soil type, crop type, N, P, K
Crop nutrient requirement [7]	22	–	required N, P, K and pH per crop

C. Data Analysis and Pre-processing

The crop dataset has no missing values and no duplicate rows. Table II gives the range of each feature. The features are on very different scales: K goes up to 205 while pH stays between 3.5 and 9.9.

TABLE II
SUMMARY OF THE CROP DATASET FEATURES

Feature	Mean	Std.	Min	Max
N	50.55	36.92	0.00	140.00
P	53.36	32.99	5.00	145.00
K	48.15	50.65	5.00	205.00
Temperature (°C)	25.62	5.06	8.83	43.68
Humidity (%)	71.48	22.26	14.26	99.98
pH	6.47	0.77	3.50	9.94
Rainfall (mm)	103.46	54.96	20.21	298.56

Fig. 2(a) shows the correlation between features. Only P and K are strongly related ($r = 0.74$); every other pair stays between -0.23 and 0.21, so each feature adds its own information. Fig. 2(b) shows how different the crops are in their nutrient needs. Apple and grapes have by far the highest P and K (around 133 and 200), cotton has the highest N (118), and crops such as mango and orange sit low on all three.

For pre-processing we encoded the crop names as integers and split the data with a stratified 80/20 split (random seed 42), giving 1,760 training and 440 test samples, 20 per crop in the test set. KNN, SVM and Logistic Regression were given standardised features (zero mean, unit variance), with the scaler fitted on the training set only. Tree models and Naive Bayes used the raw values. For the fertilizer data, soil type and crop type were one-hot encoded.

D. Algorithms and Training

Table III lists the ten algorithms and their settings. We used the library defaults apart from the settings shown, and did not tune hyperparameters. We kept AdaBoost in two forms because AdaBoost is often reported with its default settings, and those settings behave very differently on this problem (Section IV-C).

TABLE III
ALGORITHMS AND SETTINGS

Algorithm	Settings
Logistic Regression	scaled, max. 5000 iterations
Naive Bayes	Gaussian
KNN	scaled, $k = 5$
SVM	scaled, RBF kernel
Decision Tree	no depth limit
Random Forest	100 trees
XGBoost	300 trees, depth 6, learning rate 0.1
LightGBM	300 trees, 31 leaves, learning rate 0.05
AdaBoost (default)	50 decision stumps (depth 1)
AdaBoost (depth-3 trees)	200 trees of depth 3, learning rate 0.5

E. Testing and Evaluation Metrics

Each algorithm was trained on the 1,760 training samples and tested once on the 440 held-out samples. We report accuracy and the macro averages of precision, recall and F1-score over the 22 crops:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \tag{6}$$

$$F1_c = \frac{2\,\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \tag{7}$$

where TP_c, FP_c and FN_c are the true positives, false positives and false negatives for crop c. Because a single split can be lucky or unlucky, we also ran 5-fold stratified cross-validation on the training set and report its mean and standard deviation. Training time was measured on a 4-core CPU.

F. Explainable AI Module

We explained the Random Forest with TreeSHAP. The global importance of feature j is the mean absolute SHAP value over the n test samples and the $K = 22$ crop outputs:

$$I_j = \frac{1}{nK} \sum_{i=1}^{n} \sum_{k=1}^{K} \left| \phi_j^{(k)}(\mathbf{x}_i) \right|. \tag{8}$$

For each crop we also computed the importance using only that crop's own test samples and its own output, which shows what the model looks at when it recommends that crop. For a single farmer we show the SHAP values of the recommended crop next to a LIME explanation built from 2,000 perturbed samples. To see how often the two methods agree, we compared them on 100 randomly chosen test samples.

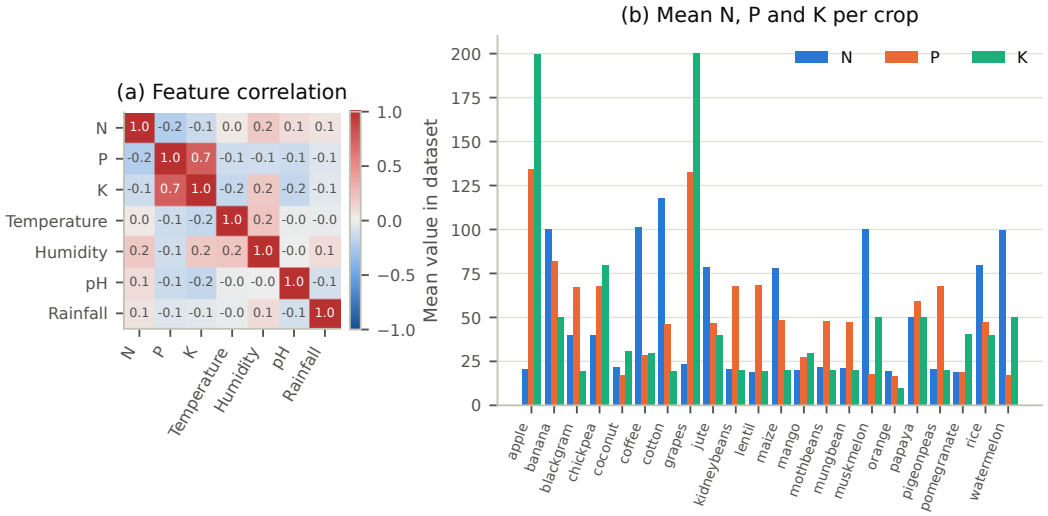


Fig. 2. Data analysis of the crop dataset. (a) Pearson correlation between the seven features. (b) Mean N, P and K for each crop.

G. Fertilizer Recommendation Module

1) Machine learning model: We trained eight algorithms to predict the fertilizer name from the eight inputs of the fertilizer dataset. With only 99 rows, a single 80/20 split would leave about 20 test rows and just one or two for the smallest classes, so we used stratified 5-fold cross-validation repeated 10 times and report the mean and standard deviation of accuracy over the 50 test folds. To see which inputs the model uses, we computed permutation importance for the Random Forest on held-out folds: the drop in accuracy when one feature's values are shuffled.

2) Nutrient-gap rule: The second part follows the approach of Harvestify [7]. For a recommended crop c with required nutrients $\mathbf{r}_c = (N, P, K)$ and the farmer's soil values $\mathbf{s}$, we compute the gap

$$\mathbf{g} = \mathbf{r}_c - \mathbf{s}, \qquad j^* = \arg\max_j |g_j|. \tag{9}$$

The nutrient j^* with the largest gap decides the advice. If $g_{j^*} > 0$ the soil is short of that nutrient and we suggest a fertilizer that supplies it: urea for N, DAP or single super phosphate (SSP) for P, and muriate of potash (MOP) for K. If $g_{j^*} < 0$ the soil already has more than the crop needs and we advise skipping that nutrient for the season.

IV. Results and Discussion

A. Experimental Setup

All experiments were written in Python 3.13 with scikit-learn 1.9.1 [21], XGBoost 3.4.1, LightGBM 4.7.0, SHAP 0.52.0 and LIME 0.2.0.1, and ran on a CPU. The random seed was 42 throughout.

B. Comparison of Algorithms

Table IV and Fig. 3 give the results on the 440 test samples. Random Forest and Naive Bayes share the top place with 99.55% accuracy; each misclassified only 2 of the 440 test samples. AdaBoost with depth-3 trees (3 errors), XGBoost (4) and LightGBM (5) follow closely. SVM, Decision Tree, KNN and Logistic Regression are a little behind, between 97.27% and 98.41%.

The differences among the top five algorithms are small. Between 99.55% and 98.86% lie only three test samples, and the cross-validation results (98.75% to 99.49%) place the same five algorithms at the top. On this dataset several algorithms work about equally well, and the choice between them should rest on other qualities. We chose Random Forest for the system. It ties for the best accuracy, trains in a quarter of a second, gives a probability for every crop so that we can show a top-3 list, and TreeSHAP explains it exactly. Naive Bayes is an equally accurate and even faster alternative when explanations are not needed.

The one clear failure is AdaBoost with default settings, at 17.50% on the test set. Its default weak learner is a stump, a tree with a single split. One split can only divide the 22 crops into two groups, so each round of boosting adds very little, and the ensemble never separates most crops. When the stump is replaced by a depth-3 tree, the same algorithm reaches 99.32%. A low AdaBoost score on this kind of problem points to the choice of weak learner, not to boosting itself.

C. Testing the Selected Model

Fig. 4 shows the confusion matrix of the Random Forest on the test set. Twenty of the 22 crops were classified perfectly. The two errors were one rice sample predicted as jute and one blackgram sample predicted as maize, which gives these four crops an F1-score between 0.974 and 0.976. Both mistakes are understandable. Rice and jute have almost the same average N, P and K (about 80, 47 and 40) and both grow in humid conditions (humidity around 80%), differing mainly in rainfall (236 mm against 175 mm). Blackgram and maize have the same average humidity (65%) and similar K.

TABLE IV
CROP RECOMMENDATION: RESULTS ON THE 440 TEST SAMPLES AND 5-FOLD CROSS-VALIDATION (CV) ON THE TRAINING SET

Algorithm	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	Errors (of 440)	CV accuracy (%)	Training time (s)
Random Forest	**99.55**	99.57	99.55	99.55	2	99.38 ± 0.61	0.24
Naive Bayes	**99.55**	99.59	99.55	99.54	2	99.49 ± 0.42	<0.01
AdaBoost (depth-3 trees)	99.32	99.37	99.32	99.30	3	98.75 ± 0.29	0.92
XGBoost	99.09	99.13	99.09	99.08	4	99.26 ± 0.14	1.02
LightGBM	98.86	98.94	98.86	98.86	5	99.03 ± 0.39	1.44
SVM (RBF)	98.41	98.56	98.41	98.40	7	97.90 ± 1.03	0.16
Decision Tree	97.95	98.06	97.95	97.94	9	98.52 ± 0.68	0.01
KNN ($k = 5$)	97.95	98.04	97.95	97.93	9	96.53 ± 1.21	<0.01
Logistic Regression	97.27	97.40	97.27	97.25	12	96.82 ± 0.66	0.04
AdaBoost (default)	17.50	11.21	17.50	8.56	363	31.93 ± 5.85	0.14

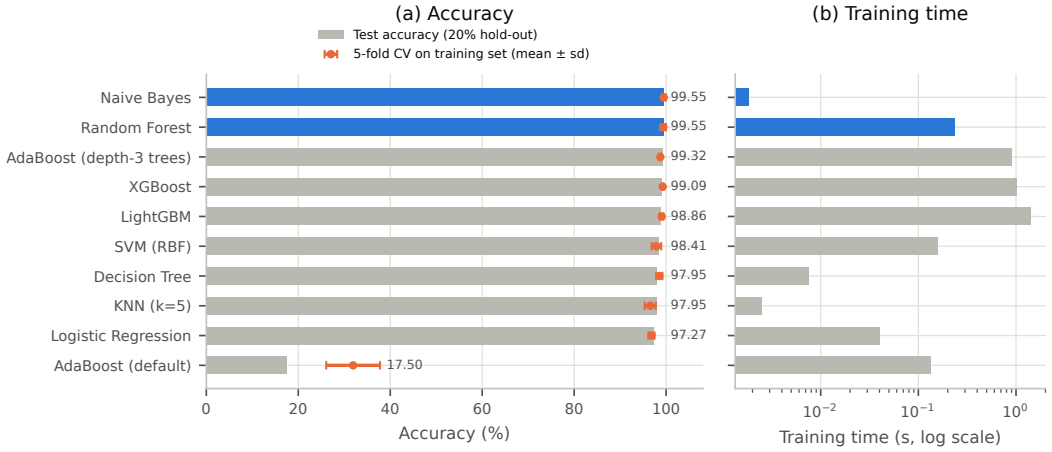


Fig. 3. Comparison of the ten algorithms. (a) Test accuracy (bars) and 5-fold cross-validation accuracy on the training set (dots, mean ± standard deviation); the two best models are highlighted. (b) Training time on a log scale.

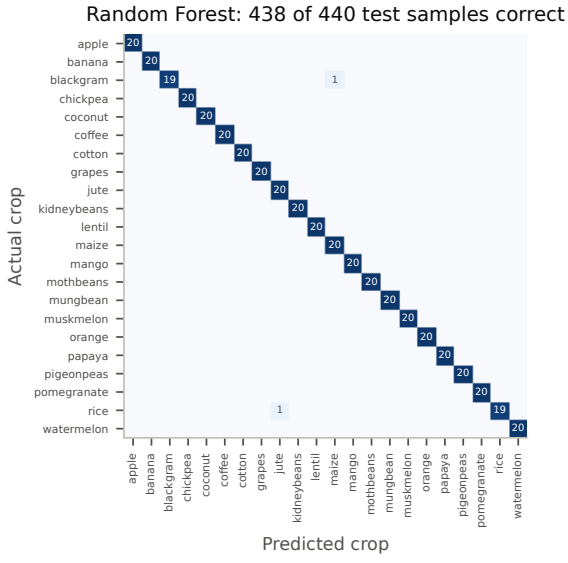


Fig. 4. Confusion matrix of the Random Forest on the 440 test samples.

Table V shows how the system responds to six farmers, taken from the test set with different crops. For all six the first recommendation matched the crop actually grown. The probabilities show how confident the model is. For F3 and F6 it is almost certain (0.99), while for F2 the first choice, moth beans, has 0.72 and pigeon peas and maize follow with 0.10 and 0.08. In such a case the top-3 list is more useful to the farmer than a single answer, since the soil and weather suit more than one crop.

D. Explainable AI: Why a Crop Is Recommended

Fig. 5(a) shows the global SHAP importance. Humidity has the largest influence (22% of the total), followed by rainfall (18%), K and N (17% each) and P (16%). Temperature (7%) and pH (3%) matter much less. The model therefore relies on a mix of weather and soil, not on one dominant feature.

The per-crop view in Fig. 5(b) is more useful to a farmer, because the reason differs from crop to crop. Rainfall is the main factor for 8 crops, including rice, jute, lentil and watermelon, and humidity for 7, including chickpea, kidney beans and mango. For apple, banana and grapes the main factor is P; for cotton and maize it is N; for orange and pomegranate it is K. These match the nutrient profiles in

TABLE V
RECOMMENDATIONS FOR SIX FARMERS FROM THE TEST SET (RANDOM FOREST; PROBABILITY IN BRACKETS)

Farmer	N	P	K	Temp.	Hum.	pH	Rain	Actual crop	1st choice	2nd choice	3rd choice
F1	18	17	31	31.7	45.2	5.7	93.8	mango	mango (0.98)	orange (0.01)	pigeon peas (0.01)
F2	22	44	24	24.3	56.3	6.0	59.0	moth beans	moth beans (0.72)	pigeon peas (0.10)	maize (0.08)
F3	3	67	24	17.0	19.9	5.5	103.3	kidney beans	kidney beans (0.99)	pigeon peas (0.01)	blackgram (0.00)
F4	34	75	24	23.5	51.3	4.8	192.3	pigeon peas	pigeon peas (0.98)	moth beans (0.02)	blackgram (0.00)
F5	76	54	45	24.3	77.6	6.2	185.0	jute	jute (0.91)	rice (0.08)	papaya (0.01)
F6	117	76	47	25.6	77.4	6.1	93.1	banana	banana (0.99)	cotton (0.01)	blackgram (0.00)

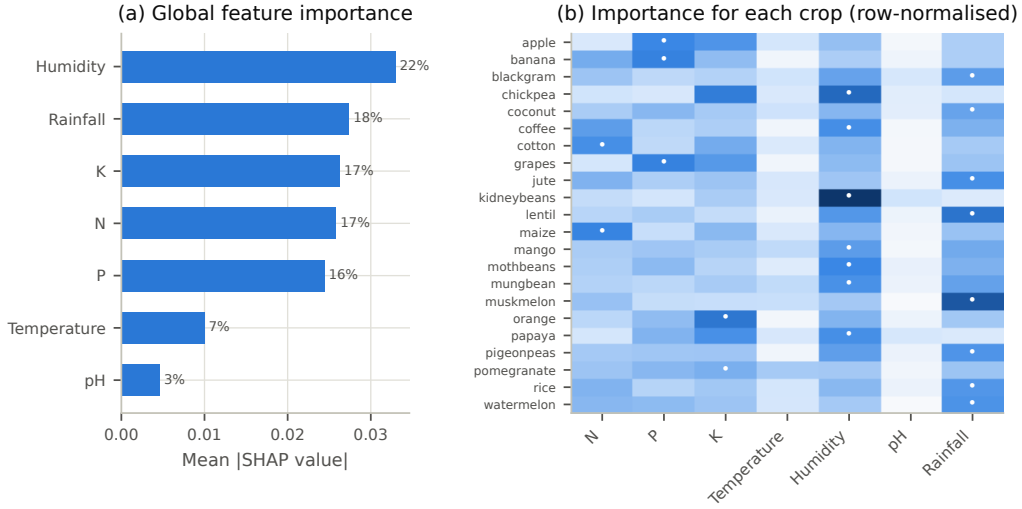


Fig. 5. SHAP explanation of the Random Forest. (a) Global importance (mean absolute SHAP value; share of the total in per cent). (b) Importance of each feature when the model recommends each crop, normalised per row; the dot marks the most important feature for that crop.

Fig. 2(b): cotton stands out by its high N, and apple and grapes by their high P and K.

Fig. 6 explains the recommendation for farmer F1, who was advised to grow mango with probability 0.98. SHAP gives the largest contributions to humidity (45.2%, contribution 0.23) and rainfall (93.8 mm, 0.20), followed by K, P and N. In plain words, the field is fairly dry and warm (31.7°C) and the soil is low in all three nutrients, which is the typical setting for mango in the data. LIME also puts humidity first and reports the rule "humidity $\leq$ 60.2", but ranks P second and rainfall fifth. The two methods agree on the overall picture but not on every detail. Over 100 test samples they named the same top feature in 66% of cases, and on average they shared 78% of their top three features. Showing the farmer only the first two or three reasons, where the methods mostly agree, is the safer choice.

E. Fertilizer Recommendation

Table VI and Fig. 7(a) compare the eight algorithms on the fertilizer dataset. XGBoost gave the highest accuracy, 97.69%, followed by Decision Tree (96.28%), Random Forest (95.07%) and Logistic Regression (94.35%). KNN (71.41%) and Naive Bayes (66.74%) did poorly. Both treat all eight inputs as equally relevant, while only three of them turn out to matter (below), and two classes have just seven rows each. The standard deviations are larger than for the crop data (3 to 12 points) because each test fold holds only about 20 rows.

TABLE VI
FERTILIZER RECOMMENDATION: ACCURACY AND MACRO F1-SCORE OVER 5-FOLD CROSS-VALIDATION REPEATED 10 TIMES

Algorithm	Accuracy (%)	F1-score (%)
XGBoost	**97.69 ± 3.20**	95.02
Decision Tree	96.28 ± 4.00	92.24
Random Forest	95.07 ± 4.43	89.22
Logistic Regression	94.35 ± 4.56	87.59
SVM (RBF)	88.09 ± 5.19	77.61
AdaBoost (depth-3 trees)	86.19 ± 12.09	74.14
KNN ($k = 5$)	71.41 ± 10.40	63.13
Naive Bayes	66.74 ± 9.59	60.58

The feature importance in Fig. 7(b) explains the result. Shuffling phosphorus lowers the Random Forest's accuracy by 0.44, nitrogen by 0.31 and potassium by 0.26, while shuffling crop type, temperature, moisture, soil type or humidity changes it by 0.006 or less. In this dataset the choice of fertilizer depends almost entirely on the soil N, P and K, which makes sense, since a fertilizer is chosen to supply the nutrients the soil lacks. In one 5-fold run XGBoost misclassified 2 of 99 samples (Fig. 7(c)): one 10-26-26 sample predicted as 14-35-14 and one 17-17-17 sample predicted as 20-20, both pairs of

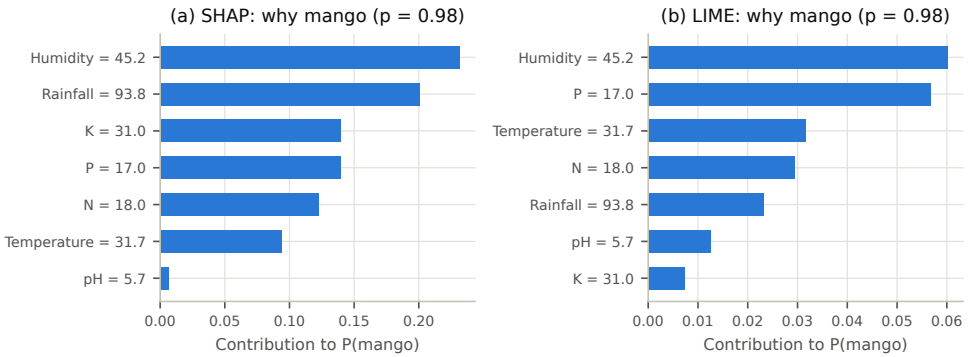


Fig. 6. Explanation of one recommendation (farmer F1, mango). (a) SHAP contribution of each input to the probability of mango. (b) LIME weights for the same input.

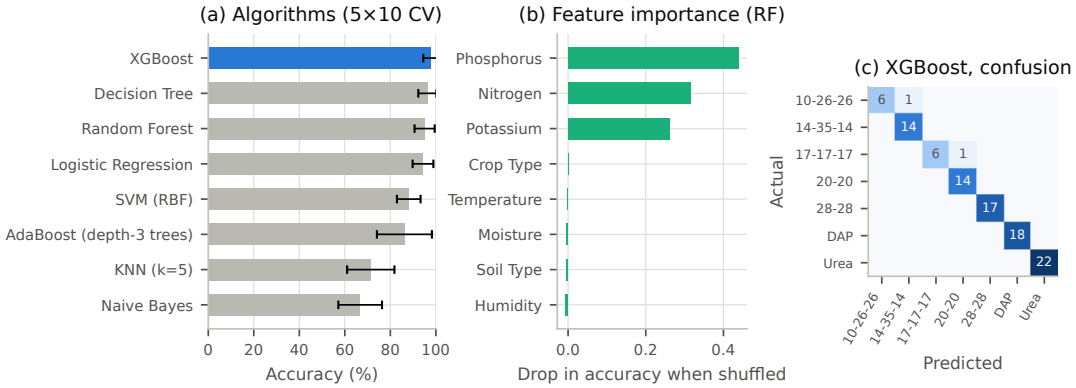


Fig. 7. Fertilizer recommendation. (a) Accuracy of the eight algorithms (mean ± standard deviation over 50 test folds). (b) Permutation importance of each input for the Random Forest, on held-out folds. (c) Confusion matrix of XGBoost over one 5-fold run.

complex fertilizers that supply all three or two of the nutrients.

Table VII applies the nutrient-gap rule to the six farmers of Table V, for the first and the second recommended crop. For the first crop the gaps are mostly small. F1's largest gap for mango is only 3 units of P, and F2's is 4 units for moth beans. This is expected, because the crop model picks the crop whose usual conditions are closest to the farmer's field. The second crop usually needs more correction: F1's soil has 21 units more K than orange needs, and F6's soil has 36 units more P than cotton needs. F3 is an exception. The soil has only 3 units of N against the 20 that kidney beans need, so the system advises adding a nitrogen fertilizer such as urea. The rule thus also tells the farmer how much the soil would have to change for a crop they prefer.

F. Discussion

Three points come out of the results. First, on this dataset the best algorithms are very close, and Random Forest, Naive Bayes, XGBoost and AdaBoost with proper weak learners all exceed 99%. Reporting only the single highest number would hide this; the cross-validation and the error counts show it clearly. Second, the explanations make the recommendations easier to check. The per-crop importance agrees with what the data show about each crop, and a farmer can see which of their own values led to the advice. SHAP and LIME do not always agree, so we treat the top two or three reasons as the explanation and do not rank every feature. Third, the fertilizer models confirm that soil N, P and K drive the fertilizer choice, and the nutrient-gap rule turns this into advice tied to the crop the farmer is about to sow.

Limitations. The crop dataset was assembled by its author, who combined typical nutrient values for each crop with weather records and notes that it should not be used for real farming decisions [7]. Our accuracies therefore show how well the algorithms learn this dataset; they are not yet a measure of performance on farmers' fields, which needs testing with real soil test reports. The fertilizer dataset has only 99 rows, and two of its classes have seven rows each. Neither dataset states the units of N, P and K, so the nutrient-gap advice names a nutrient and a direction, not a quantity in kg/ha. Finally, all algorithms used default settings; tuning could change the order among the closely matched top models.

V. Conclusion and Future Work

We built a crop recommendation system that recommends a crop, explains the recommendation and suggests a fertilizer. Of the ten algorithms we tested, Random Forest and Naive Bayes were the most accurate, both at 99.55% on 440 test

TABLE VII
NUTRIENT-GAP ADVICE FOR THE SIX FARMERS (GAP = CROP REQUIREMENT − SOIL VALUE; THE LARGEST GAP DECIDES THE ADVICE)

Farmer	Soil N-P-K	Rank	Crop	Required N-P-K	Largest gap	Advice
F1	18-17-31	1	mango	20-20-30	P low by 3	add phosphate (DAP or SSP)
		2	orange	20-10-10	K high by 21	skip potash this season
F2	22-44-24	1	moth beans	20-40-20	P high by 4	skip phosphate this season
		2	pigeon peas	20-60-20	P low by 16	add phosphate (DAP or SSP)
F3	3-67-24	1	kidney beans	20-60-20	N low by 17	add nitrogen (urea)
		2	pigeon peas	20-60-20	N low by 17	add nitrogen (urea)
F4	34-75-24	1	pigeon peas	20-60-20	P high by 15	skip phosphate this season
		2	moth beans	20-40-20	P high by 35	skip phosphate this season
F5	76-54-45	1	jute	80-40-40	P high by 14	skip phosphate this season
		2	rice	80-40-40	P high by 14	skip phosphate this season
F6	117-76-47	1	banana	100-75-50	N high by 17	skip nitrogen this season
		2	cotton	120-40-20	P high by 36	skip phosphate this season

samples, and the five best were within three test samples of each other. AdaBoost failed with its default stumps (17.50%) but reached 99.32% with depth-3 trees. SHAP showed that humidity and rainfall have the largest overall influence, with N, P and K close behind, and that the deciding factor changes from crop to crop. For fertilizer recommendation, XGBoost reached 97.69% and the soil N, P and K values carried almost all of the importance.

In future work we plan to test the system on real Soil Health Card reports combined with district rainfall records, so that the recommendations can be checked against what farmers actually grow. We also plan to convert the nutrient gap into fertilizer quantities in kg/ha using soil test based dose recommendations, to tune the hyperparameters of the strongest models, and to build a mobile application that gives the explanation in the farmer's own language.

REFERENCES

[1] K. G. Liakos, P. Busato, D. Moshou, S. Pearson, and D. Bochtis, "Machine learning in agriculture: A review," *Sensors*, vol. 18, no. 8, p. 2674, 2018.

[2] T. van Klompenburg, A. Kassahun, and C. Catal, "Crop yield prediction using machine learning: A systematic literature review," *Computers and Electronics in Agriculture*, vol. 177, p. 105709, 2020.

[3] R. Kumar, M. P. Singh, P. Kumar, and J. P. Singh, "Crop selection method to maximize crop yield rate using machine learning technique," in *Proc. Int. Conf. Smart Technologies and Management for Computing, Communication, Controls, Energy and Materials (ICSTM)*, 2015, pp. 138–145.

[4] S. Pudumalar, E. Ramanujam, R. H. Rajashree, C. Kavya, T. Kiruthika, and J. Nisha, "Crop recommendation system for precision agriculture," in *Proc. 8th Int. Conf. Advanced Computing (ICoAC)*, 2017, pp. 32–36.

[5] Z. Doshi, S. Nadkarni, R. Agrawal, and N. Shah, "AgroConsultant: Intelligent crop recommendation system using machine learning algorithms," in *Proc. 4th Int. Conf. Computing Communication Control and Automation (ICCUBEA)*, 2018, pp. 1–6.

[6] M. Ryo, "Explainable artificial intelligence and interpretable machine learning for agricultural data analysis," *Artificial Intelligence in Agriculture*, vol. 6, pp. 257–265, 2022.

[7] A. Ingle (Gladiator07), "Harvestify: Crop and fertilizer recommendation and plant disease detection," GitHub repository, 2023. [Online]. Available: https://github.com/Gladiator07/Harvestify

[8] A. Ingle, "Crop recommendation dataset," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset

[9] "Fertilizer prediction," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/gdabhishek/fertilizer-prediction

[10] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.

[11] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[12] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[13] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[14] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[15] J. Zhu, H. Zou, S. Rosset, and T. Hastie, "Multi-class AdaBoost," *Statistics and Its Interface*, vol. 2, no. 3, pp. 349–360, 2009.

[16] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[17] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.

[18] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765–4774.

[19] S. M. Lundberg *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.

[20] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.

[21] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.